

ECE 220 Computer Systems & Programming

Lecture 8 — Implementing Functions in C: the Run-Time Stack

September 17, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

A Concrete Example: Factorial

```
#include <stdio.h>
int Fact(int n);

int main() {
    int number, answer;
    printf("Enter a number: ");
    scanf("%d", &number);
    answer = Fact(number);
    printf("factorial of %d is %d\n", number, answer);
    return 0;
}

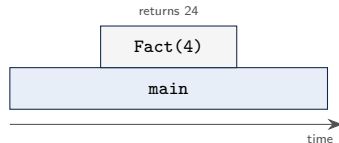
int Fact(int n) {
    int i, result = 1;
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;
}
```

A Concrete Example: Factorial

```
#include <stdio.h>
int Fact(int n);

int main() {
    int number, answer;
    printf("Enter a number: ");
    scanf("%d", &number);
    answer = Fact(number);
    printf("factorial of %d is %d\n", number, answer);
    return 0;
}

int Fact(int n) {
    int i, result = 1;
    for (i = 1; i <= n; i++)
        result = result * i;
    return result;
}
```



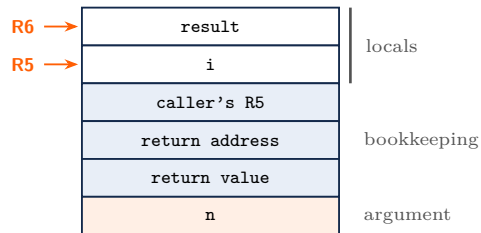
Suppose you type 4. `main` waits while `Fact` runs, then gets 24 back.

Where do `number`, `answer`, `n`, `i` and `result` live — and how does `Fact` find its way back to `main`?

The Activation Record (Stack Frame)

Every live call gets one. For `Fact`, with argument `n` and locals `i` and `result`:

- **R5**, the frame pointer, marks the first local; **R6**, the stack pointer, the top
- **locals**: `i` at $R5 + ____$, `result` at $R5 - ____$
- **bookkeeping**: caller's R5, return address, return value
- **argument**: `n` at $R5 + ____$, past the three bookkeeping slots

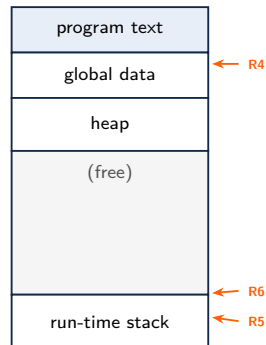


Lower addresses at the top: the stack grows up the page.

Three Registers Run Everything

- **R4 — global pointer:** first global variable; globals at fixed offsets from R4
- **R5 — frame pointer:** first *local* of the currently running function
- **R6 — stack pointer:** top-most occupied slot of the run-time stack

A local variable lives at $R5 + \underline{\hspace{2cm}}$



The Symbol Table

The compiler records every identifier: **name**, **type**, **location (as an offset)**, **scope**.

```
int inGlobal;
int outGlobal;
int main(){
    int x, y, z; ...
}
int dummy(int in1, int in2){
    int a, b, c; ...
}
```

Name	Type	Offset	Scope
inGlobal	int	0 (R4)	global
outGlobal	int	1 (R4)	global
x	int	0 (R5)	main
y	int	-1	main
z	int	-2	main
a	int	_____	_____
b	int	_____	
c	int	_____	

Build-Up, by Hand

	contents	what it is
x3FF7		
x3FF8		
x3FF9		
x3FFA		
x3FFB		
x3FFC		
x3FFD		
x3FFE		
x3FFF	<code>answer = ?</code>	main: local
x4000	<code>number = 4</code>	main: local

Snapshot: `Fact(4)` is about to return.

- Push `main`'s call to `Fact`, one phase at a time
- Draw R5 and R6, and bracket each record
- `Fact` reads `n` with `LDR R1, R5, #_____`
- The 4 at x3FFE was pushed by _____

Tear-Down, by Hand

	contents	what it is
x3FF7		
x3FF8		
x3FF9		
x3FFA		
x3FFB		
x3FFC		
x3FFD		
x3FFE		
x3FFF	<code>answer =</code>	main: local
x4000	<code>number = 4</code>	main: local

Snapshot: `Fact` has returned, and `main` has finished its tear-down.

- The 24 travels: `result` → _____ → `answer`
- R6 is back at _____
- What is left in x3FF9–x3FFE?

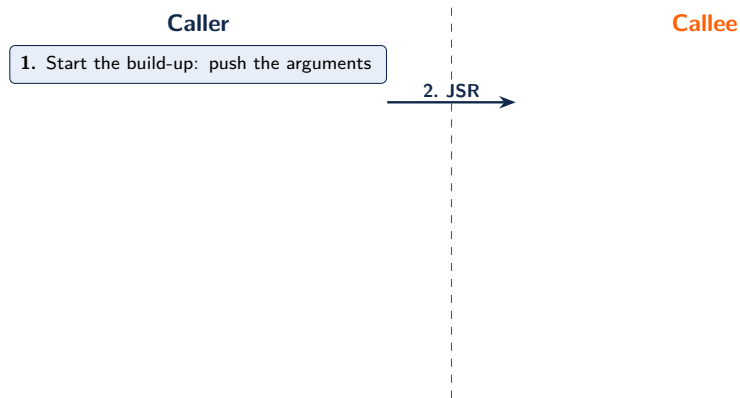
Eight Phases of a Call

Caller

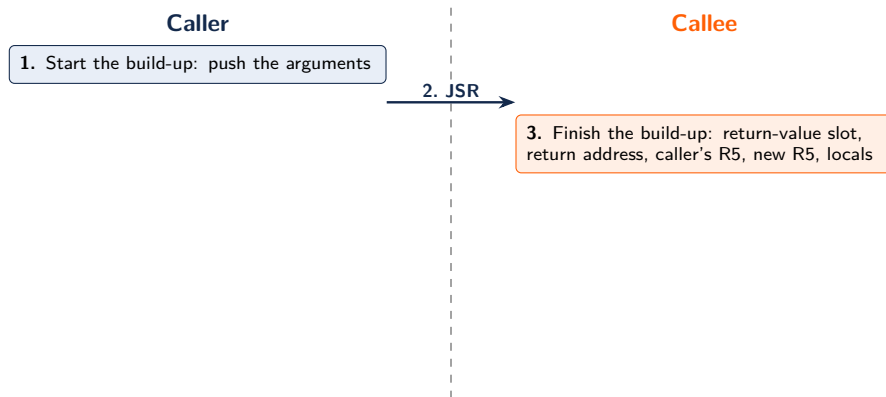
1. Start the build-up: push the arguments

Callee

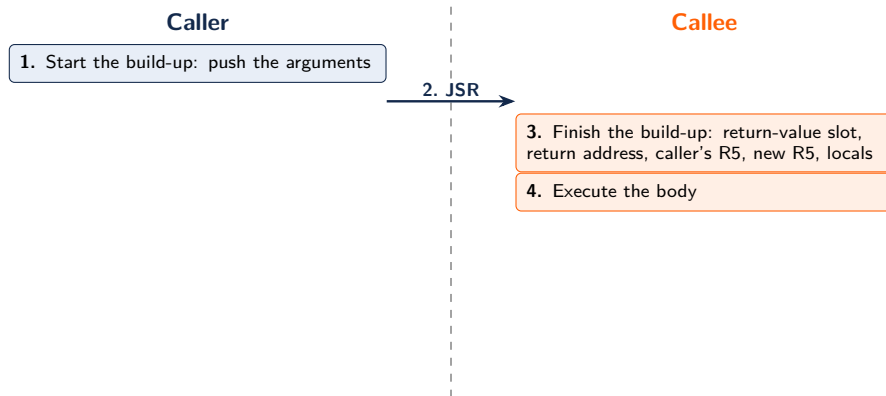
Eight Phases of a Call



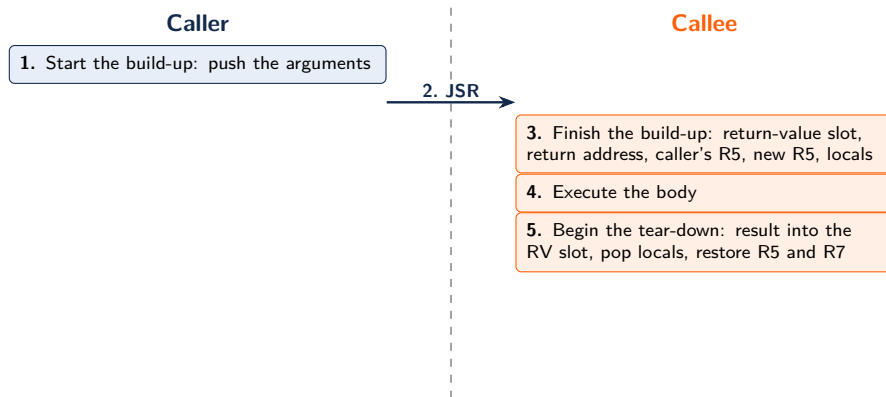
Eight Phases of a Call



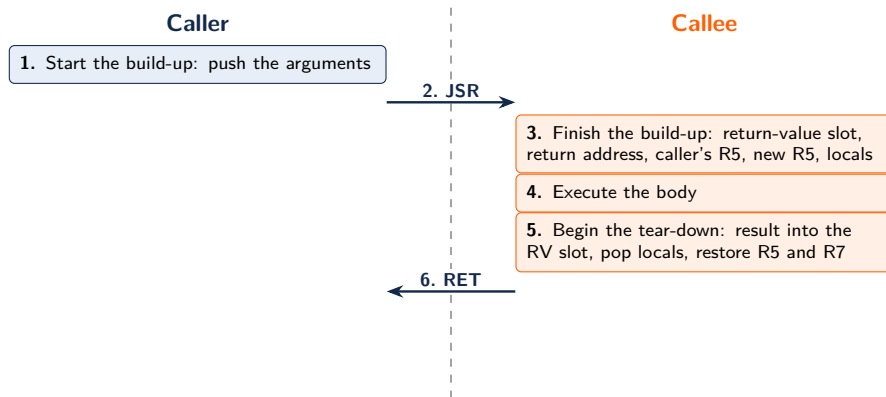
Eight Phases of a Call



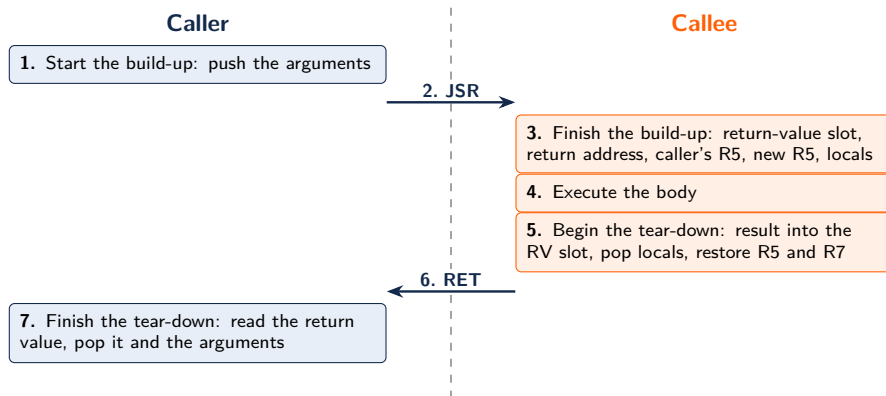
Eight Phases of a Call



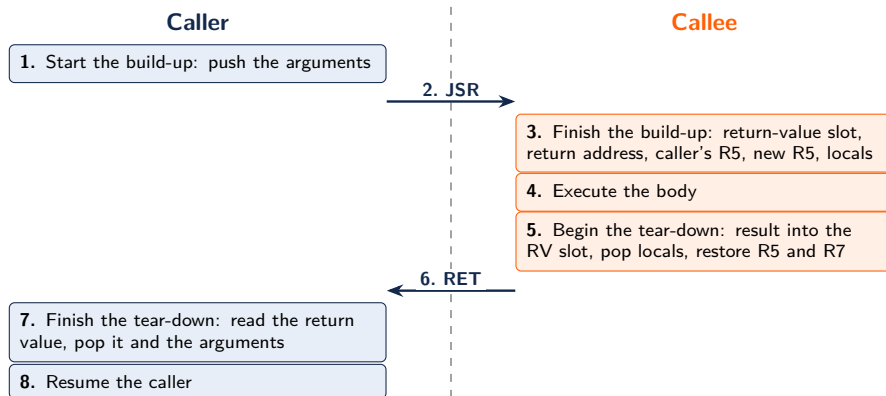
Eight Phases of a Call



Eight Phases of a Call



Eight Phases of a Call



In the example, `main` runs 1, 2, 7 and 8, and `Fact` runs 3–6. Each side cleans up what it pushed.

The Call in LC-3: the Caller's Side

```
.ORIG x3000
    LD    R6, USER_STACK    ; R6: stack pointer
    ADD   R5, R6, #-1        ; R5: frame pointer
    ADD   R6, R6, #-2        ; locals: number, answer
    AND   R0, R0, #0
    ADD   R0, R0, #4
    STR   R0, R5, #0         ; number = 4
; ---- 1. caller setup: push the argument ----
; ---- 2. pass control to the callee ----
; ---- 7. caller teardown ----
; load the return value (where is it?)
; perform the assignment: answer = ...
; pop the return value and the argument
; ---- 8. resume the caller ----
    HALT
```

- Where is the return value when JSR comes back?

- Why pop two slots at the end?

The Call in LC-3: the Callee's Side

```
; ==== int Fact(int n) ====  
; ---- 3. callee setup ----  
FACT ; reserve the return-value slot  
  
; push the return address (which register?)  
  
; push the caller's frame pointer  
  
; set the new frame pointer; make room for locals  
  
; ---- 4. execute (given in callconv.asm) ----  
; ...  
; ---- 5. callee teardown ----  
; copy the result into the RV slot  
  
; pop the locals (one instruction!)  
  
; restore caller's R5, then R7  
  
; ---- 6. return to the caller ----
```

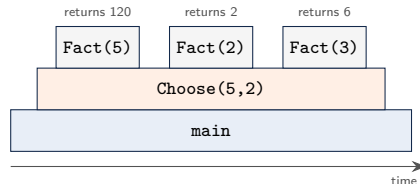
- The argument `n` is read with `LDR R1, R5, #_____`
- `ADD R6, R5, #1` pops *all* locals at once — how does it know how many?

Nesting: 5 Choose 2

```
int Fact(int n) {  
    int i, result = 1;  
    for (i = 1; i <= n; i++)  
        result = result * i;  
    return result;  
}  
int Choose(int n, int k) {  
    int ways;  
    ways = Fact(n);  
    ways = ways / Fact(k);  
    ways = ways / Fact(n - k);  
    return ways;  
}  
int main() {  
    int answer;  
    answer = Choose(5, 2);  
    printf("5 choose 2 = %d\n", answer);  
    return 0;  
}
```

Nesting: 5 Choose 2

```
int Fact(int n) {  
    int i, result = 1;  
    for (i = 1; i <= n; i++)  
        result = result * i;  
    return result;  
}  
int Choose(int n, int k) {  
    int ways;  
    ways = Fact(n);  
    ways = ways / Fact(k);  
    ways = ways / Fact(n - k);  
    return ways;  
}  
int main() {  
    int answer;  
    answer = Choose(5, 2);  
    printf("5 choose 2 = %d\n", answer);  
    return 0;  
}
```



At most three calls deep. **Fact** runs three times while **Choose** waits; **Choose** returns 10.

Now three records sit on the stack at once, and **Fact's is pushed and popped three times.**

Nested Build-Up, by Hand

	contents	what it is
x3FF4		
x3FF5		
x3FF6		
x3FF7		
x3FF8		
x3FF9		
x3FFA		
x3FFB		
x3FFC		
x3FFD		
x3FFE		
x3FFF		
x4000	answer = ?	main: local

Snapshot: `Fact(5)` is about to return.

- Push `main`'s call to `Choose`, then `Choose`'s call to `Fact(5)`
- Draw R5 and R6, and bracket each record
- One `Fact` call pushes _____ words
- The 5 at x3FF9 was pushed by _____

Nested Tear-Down, by Hand

	contents	what it is
x3FF4		
x3FF5		
x3FF6		
x3FF7		
x3FF8		
x3FF9		
x3FFA		
x3FFB		
x3FFC		
x3FFD		
x3FFE		
x3FFF		
x4000	<code>answer = ?</code>	<code>main: local</code>

Snapshot: all three `Fact` calls are done, and `Choose` is about to return.

- Each `Fact` call used

- What is left in x3FF4–x3FF9?

- When `main` resumes: `R6 = _____`,
`answer = _____`

swap, Explained

```
void swap(int x, int y) {  
    int temp;  
    temp = x;  
    x = y;  
    y = temp;  
}  
  
int main() {  
    int x = 2, y = 3;  
    swap(x, y);  
    printf("x = %d, y = %d\n", x, y);  
    return 0;  
}
```

- main's `x`, `y` live in _____
- swap's `x`, `y` are the argument slots of _____
- swap dutifully exchanges _____ — then its frame is popped and they vanish

To change main's `x`, swap needs main's address for `x` — not its value. Next week: pointers.

Summary & What's Next

Today

- **One activation record per live call:** locals from R5 down; bookkeeping (caller's R5, return address, return value); arguments from R5+4
- **Push and pop:** `main` → `Fact` by hand, then nested through `Choose`; popping moves R6 but erases nothing
- **Eight phases:** the caller starts the build-up and finishes the tear-down; the callee does everything in between; `JSR` and `RET` carry control
- **swap** failed because its arguments are copies living in its own record

Next lecture

- Pointers and arrays — passing *addresses*, and what `&` has meant all along

Code from today: `fact.c`, `callconv.asm` (the same call, runnable), `choose.c`, `swap.c`.